

1. What is file handling in Python? Name different file modes.

File handling refers to the process of performing operations on a file, such as creating, opening, reading, writing and closing it through a programming interface. It involves managing the data flow between the program and the file system on the storage device, ensuring that data is handled safely and efficiently.

Why do we need File Handling

- To store data permanently, even after the program ends.
- To access external files like .txt, .csv, .json, etc.
- To process large files efficiently without using much memory.
- To automate tasks like reading configs or saving outputs.

File modes: When working with files in Python, the file mode tells Python what kind of operations (read, write, etc.) you want to perform on the file. You specify the mode as the second argument to the `open()` function.

Mode	Description
'r'	Read-only. Raises I/O error if file doesn't exist.
'r+'	Read and write. Raises I/O error if the file does not exist.
'w'	Write-only. Overwrites file if it exists, else creates a new one.
'w+'	Read and write. Overwrites file or creates new one.
'a'	Append-only. Adds data to end. Creates file if it doesn't exist.
'a+'	Read and append. Pointer at end. Creates file if it doesn't exist.
'rb'	Read in binary mode. File must exist.

'rb+'	Read and write in binary mode. File must exist.
'wb'	Write in binary. Overwrites or creates new.
'wb+'	Read and write in binary. Overwrites or creates new.
'ab'	Append in binary. Creates file if not exist.
'ab+'	Read and append in binary. Creates file if it does not exist.

2. What is the difference between read(), readline(), and readlines()?

1. read()

- Reads the entire file content at once
- Returns data as one single string
- Not memory-efficient for large files (Suitable for small files)

2. readline()

- Reads one line at a time
- Returns a single string
- Useful for line-by-line processing
- Large files

3. readlines()

- Reads all lines at once
- Returns a list of strings (each line is one element)
- Uses more memory than readline()
- Uses When you need all lines as a list

3. What is exception hierarchy in Python?

The exception hierarchy in Python is the structured way in which all built-in exceptions are organized into parent–child classes. It shows how exceptions inherit from one another, with BaseException at the top. An exception is an

error that occurs during the execution of a program and interrupts the normal flow of the program.

Why exception hierarchy is important

- Helps understand which exceptions are related
- Allows catching multiple related errors using a parent class
- Prevents catching overly broad exceptions accidentally
- Improves error handling and debugging

4. Difference between try-except and try-except-finally

`try-except` handles errors, while `try-except-finally` handles errors and ensures execution of cleanup code regardless of exceptions.

1. try-except

- Used to handle exceptions (errors) that occur in a program
- Code inside except runs only if an exception occurs
- Does not guarantee execution of cleanup code

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

2. try-except-finally

- Used to handle exceptions and execute cleanup code
- finally block always executes, whether an exception occurs or not
- Commonly used for closing files, database connections, or releasing resources

```
try:
    file = open("data.txt", "r")
    print(file.read())
except FileNotFoundError:
    print("File not found")
finally:
    file.close()
```

5. What is the purpose of the with statement?

The with statement is used to simplify exception handling by automatically closing or releasing resources after use.

- Replaces long try-except-finally blocks with cleaner syntax.
- Improves readability by reducing unnecessary boilerplate code.

Key advantages

- No need for try-finally
- Prevents resource leaks
- Improves readability
- Safer exception handling

```
without "with"

file = open("data.txt", "r")
data = file.read()
file.close()

with "with"

with open("data.txt", "r") as file:
    data = file.read()
```

6. What are iterators in Python?

An iterator in Python is an object used to traverse through all the elements of a collection (like lists, tuples or dictionaries) one element at a time. It follows the iterator protocol, which involves two key methods:

- **`__iter__()`**: Returns the iterator object itself.
- **`__next__()`**: Returns the next value from the sequence. Raises `StopIteration` when the sequence ends.

```
s = "GFG"
it = iter(s)

print(next(it))
print(next(it))
print(next(it))
```

✓ 0.0s

G
F
G

7. What are generators and how are they different from functions?

Generator functions in python make it easy to generate data, allowing for efficient memory utilization and lazy evaluation. That is totally different from normal functions, which run completely and return a single value, but generator functions simply employ the 'yield' keyword to generate values one at a time as the condition is stated. Because of this difference, generator function are suited for working with enormous datasets or infinite sequences.

```
def squares_func(n):  
    result = []  
    for i in range(n):  
        result.append(i * i)  
    return result
```

✓ 0.0s

Example of a generator

```
def squares_gen(n):  
    for i in range(n):  
        yield i * i
```

8. What is *args and **kwargs?

In Python, *args and **kwargs are used to allow functions to accept an arbitrary number of arguments. These features provide great flexibility when designing functions that need to handle a varying number of inputs.

1. *args (Non-keyword / positional arguments)

- *args allows a function to accept any number of positional arguments
- Arguments are stored as a tuple

2. **kwargs (Keyword arguments)

- **kwargs allows a function to accept any number of keyword arguments

- Arguments are stored as a dictionary

```
# *args example
def fun(*args):
    return sum(args)

print(fun(5, 10, 15))
```

✓ 0.0s

30

```
# **kwargs example
def fun(**kwargs):
    for k, val in kwargs.items():
        print(k, val)

fun(a=1, name="Resya", age=22, course="Data Science")
```

✓ 0.0s

a 1
name Resya
age 22
course Data Science

9. What is type casting? Give examples.

Type Casting is the method to convert the Python variable datatype into a certain data type in order to perform the required operation by users.

Type Casting in Python:

1. Implicit Type Casting

- Python automatically converts one data type to another
- Usually from lower to higher precision

```
x = 10      # int
y = 2.5     # float
z = x + y   # int → float automatically
print(z)    # 12.5
```

2. Explicit Type Casting

- Programmer manually converts data type
- Uses built-in functions like `int()`, `float()`, `str()`

```
a = 5.9
n = int(a)

print(n)
print(type(n))
```

✓ 0.0s

5
<class 'int'>

10. What are built-in functions in Python? Name any five.

Built-in functions are predefined functions provided by Python that are available for use without importing any module. Python provides a lot of built-in functions that ease the writing of code.

- **print()** – Displays output on the screen
- **len()** – Returns the length of an object
- **type()** – Returns the data type of a variable
- **input()** – Takes input from the user
- **sum()** – Returns the sum of elements in an iterable